

Bucket Sort and Order Statistics

Dr. Bugra Caskurlu¹

¹School of Computing
New Uzbekistan University

October, 2025

Bucket Sort

Restrictions on the Input

- **Input:** n numbers generated by a random process that distributes elements *uniformly* over the interval $[0, 1)$.

Bucket Sort

Restrictions on the Input

- **Input:** n numbers generated by a random process that distributes elements *uniformly* over the interval $[0, 1)$.
- **Output:** n numbers given as the input but in sorted order.

Bucket Sort

Restrictions on the Input

- **Input:** n numbers generated by a random process that distributes elements *uniformly* over the interval $[0, 1)$.
- **Output:** n numbers given as the input but in sorted order.

Algorithm Idea

- Divide the interval $[0, 1)$ into n equal-sized subintervals (**buckets**), and then distribute the n input numbers into buckets.

Bucket Sort

Restrictions on the Input

- **Input:** n numbers generated by a random process that distributes elements *uniformly* over the interval $[0, 1)$.
- **Output:** n numbers given as the input but in sorted order.

Algorithm Idea

- Divide the interval $[0, 1)$ into n equal-sized subintervals (**buckets**), and then distribute the n input numbers into buckets.
- Sort the numbers in each bucket using insertion sort.

Bucket Sort

Restrictions on the Input

- **Input:** n numbers generated by a random process that distributes elements *uniformly* over the interval $[0, 1)$.
- **Output:** n numbers given as the input but in sorted order.

Algorithm Idea

- Divide the interval $[0, 1)$ into n equal-sized subintervals (**buckets**), and then distribute the n input numbers into buckets.
- Sort the numbers in each bucket using insertion sort.
- Go through the buckets in order, listing the elements in each.

BucketSort Algorithm

Pseudocode

BUCKET-SORT(A)

$n = \text{length}[A];$

for $i = 1$ **to** n

 insert $A[i]$ into list $B[\text{floor}(n \cdot A[i])]$

for $i = 0$ **to** $n-1$

 sort list $B[i]$ with insertion sort

 concatenate the lists $B[0], B[1], \dots, B[n-1]$ in order

BucketSort Algorithm

Pseudocode

```
BUCKET-SORT(A)
  n = length[A];
  for i = 1 to n
    insert A[i] into list B[floor(n·A[i])]

  for i = 0 to n-1
    sort list B[i] with insertion sort

  concatenate the lists B[0], B[1], ..., B[n - 1] in order
```

Correctness

- Consider two elements $A[i]$ and $A[j]$. Assume $A[i] \leq A[j]$. Since $\lfloor n \cdot A[i] \rfloor \leq \lfloor n \cdot A[j] \rfloor$, either $A[i]$ is placed into the same bucket as $A[j]$ or into a bucket with a lower index.

BucketSort Algorithm

Pseudocode

BUCKET-SORT (A)

$n = \text{length}[A];$

for $i = 1$ **to** n

 insert $A[i]$ into list $B[\text{floor}(n \cdot A[i])]$

for $i = 0$ **to** $n-1$

 sort list $B[i]$ with insertion sort

 concatenate the lists $B[0], B[1], \dots, B[n-1]$ in order

Correctness

- Consider two elements $A[i]$ and $A[j]$. Assume $A[i] \leq A[j]$. Since $\lfloor n \cdot A[i] \rfloor \leq \lfloor n \cdot A[j] \rfloor$, either $A[i]$ is placed into the same bucket as $A[j]$ or into a bucket with a lower index.
- If $A[i]$ and $A[j]$ are in the same bucket then the for loop puts them in the order. If they are in different buckets concatenation phase puts them in the correct order.

Time-Complexity Analysis

Best and Worst Cases

- **Best Case:** If each bucket has 1 element, second for loop and thus the algorithm takes $\Theta(n)$ -time.

Time-Complexity Analysis

Best and Worst Cases

- **Best Case:** If each bucket has 1 element, second for loop and thus the algorithm takes $\Theta(n)$ -time.
- **Worst Case:** If all elements are in the same bucket then running time is $\Theta(n^2)$.

Time-Complexity Analysis

Best and Worst Cases

- **Best Case:** If each bucket has 1 element, second for loop and thus the algorithm takes $\Theta(n)$ -time.
- **Worst Case:** If all elements are in the same bucket then running time is $\Theta(n^2)$.
- **Average Case:** Note that running time is $\Theta(n)$ plus the **expected** time taken by the n calls to insertion sort.

Time-Complexity Analysis

Best and Worst Cases

- **Best Case:** If each bucket has 1 element, second for loop and thus the algorithm takes $\Theta(n)$ -time.
- **Worst Case:** If all elements are in the same bucket then running time is $\Theta(n^2)$.
- **Average Case:** Note that running time is $\Theta(n)$ plus the **expected** time taken by the n calls to insertion sort.

Analysis

- Let n_i be the random variable denoting the number of elements placed in bucket $B[i]$.

Time-Complexity Analysis

Best and Worst Cases

- **Best Case:** If each bucket has 1 element, second for loop and thus the algorithm takes $\Theta(n)$ -time.
- **Worst Case:** If all elements are in the same bucket then running time is $\Theta(n^2)$.
- **Average Case:** Note that running time is $\Theta(n)$ plus the **expected** time taken by the n calls to insertion sort.

Analysis

- Let n_i be the random variable denoting the number of elements placed in bucket $B[i]$.
- The running time of bucket sort is $T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$.

Time-Complexity Analysis

Best and Worst Cases

- **Best Case:** If each bucket has 1 element, second for loop and thus the algorithm takes $\Theta(n)$ -time.
- **Worst Case:** If all elements are in the same bucket then running time is $\Theta(n^2)$.
- **Average Case:** Note that running time is $\Theta(n)$ plus the **expected** time taken by the n calls to insertion sort.

Analysis

- Let n_i be the random variable denoting the number of elements placed in bucket $B[i]$.
- The running time of bucket sort is $T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$.
- Thus, average case running time of bucket sort is
$$\mathbb{E}[T(n)] = \mathbb{E}[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)] = \Theta(n) + \sum_{i=0}^{n-1} \mathbb{E}[O(n_i^2)] = \Theta(n) + \sum_{i=0}^{n-1} O(\mathbb{E}[n_i^2]).$$

Time-Complexity Analysis

Best and Worst Cases

- **Best Case:** If each bucket has 1 element, second for loop and thus the algorithm takes $\Theta(n)$ -time.
- **Worst Case:** If all elements are in the same bucket then running time is $\Theta(n^2)$.
- **Average Case:** Note that running time is $\Theta(n)$ plus the **expected** time taken by the n calls to insertion sort.

Analysis

- Let n_i be the random variable denoting the number of elements placed in bucket $B[i]$.
- The running time of bucket sort is $T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$.
- Thus, average case running time of bucket sort is
$$\mathbb{E}[T(n)] = \mathbb{E}[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)] = \Theta(n) + \sum_{i=0}^{n-1} \mathbb{E}[O(n_i^2)] = \Theta(n) + \sum_{i=0}^{n-1} O(\mathbb{E}[n_i^2]).$$
- **Claim:** $\mathbb{E}[n_i^2] = 2 - \frac{1}{n} \forall i \in \{0, 1, \dots, n-1\}$.

Time-Complexity Analysis

Best and Worst Cases

- **Best Case:** If each bucket has 1 element, second for loop and thus the algorithm takes $\Theta(n)$ -time.
- **Worst Case:** If all elements are in the same bucket then running time is $\Theta(n^2)$.
- **Average Case:** Note that running time is $\Theta(n)$ plus the **expected** time taken by the n calls to insertion sort.

Analysis

- Let n_i be the random variable denoting the number of elements placed in bucket $B[i]$.
- The running time of bucket sort is $T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$.
- Thus, average case running time of bucket sort is

$$\mathbb{E}[T(n)] = \mathbb{E}[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)] = \Theta(n) + \sum_{i=0}^{n-1} \mathbb{E}[O(n_i^2)] = \Theta(n) + \sum_{i=0}^{n-1} O(\mathbb{E}[n_i^2]).$$
- **Claim:** $\mathbb{E}[n_i^2] = 2 - \frac{1}{n} \forall i \in \{0, 1, \dots, n-1\}$.
- Hence, $\mathbb{E}[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(1) = \Theta(n)$.

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right]$

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] = \mathbb{E}\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right]$.

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] = \mathbb{E}\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right]$.
- $= \mathbb{E}\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} X_{ij} X_{ik}\right] =$

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] = \mathbb{E}\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right]$.
- $= \mathbb{E}\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} X_{ij} X_{ik}\right] = \sum_{j=1}^n \mathbb{E}[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} \mathbb{E}[X_{ij} X_{ik}]$.

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] = \mathbb{E}\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right]$.
- $= \mathbb{E}\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} X_{ij} X_{ik}\right] = \sum_{j=1}^n \mathbb{E}[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} \mathbb{E}[X_{ij} X_{ik}]$.
- What is $\mathbb{E}[X_{ij}^2]$?

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] = \mathbb{E}\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right]$.
- $= \mathbb{E}\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} X_{ij} X_{ik}\right] = \sum_{j=1}^n \mathbb{E}[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} \mathbb{E}[X_{ij} X_{ik}]$.
- What is $\mathbb{E}[X_{ij}^2]$? $\mathbb{E}[X_{ij}^2] = 1 \cdot \frac{1}{n} + 0 \cdot \left(1 - \frac{1}{n}\right) = \frac{1}{n} \forall i, j$.

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] = \mathbb{E}\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right]$.
- $= \mathbb{E}\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} X_{ij} X_{ik}\right] = \sum_{j=1}^n \mathbb{E}[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} \mathbb{E}[X_{ij} X_{ik}]$.
- What is $\mathbb{E}[X_{ij}^2]$? $\mathbb{E}[X_{ij}^2] = 1 \cdot \frac{1}{n} + 0 \cdot \left(1 - \frac{1}{n}\right) = \frac{1}{n} \forall i, j$.
- What is $\mathbb{E}[X_{ij} X_{ik}]$ for $j \neq k$?

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] = \mathbb{E}\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right]$.
- $= \mathbb{E}\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} X_{ij} X_{ik}\right] = \sum_{j=1}^n \mathbb{E}[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} \mathbb{E}[X_{ij} X_{ik}]$.
- What is $\mathbb{E}[X_{ij}^2]$? $\mathbb{E}[X_{ij}^2] = 1 \cdot \frac{1}{n} + 0 \cdot \left(1 - \frac{1}{n}\right) = \frac{1}{n} \forall i, j$.
- What is $\mathbb{E}[X_{ij} X_{ik}]$ for $j \neq k$?
- **Lemma:** Let X and Y be two **independent** random variables. Then $\mathbb{E}[X \cdot Y] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] = \mathbb{E}\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right]$.
- $= \mathbb{E}\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} X_{ij} X_{ik}\right] = \sum_{j=1}^n \mathbb{E}[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} \mathbb{E}[X_{ij} X_{ik}]$.
- What is $\mathbb{E}[X_{ij}^2]$? $\mathbb{E}[X_{ij}^2] = 1 \cdot \frac{1}{n} + 0 \cdot \left(1 - \frac{1}{n}\right) = \frac{1}{n} \forall i, j$.
- What is $\mathbb{E}[X_{ij} X_{ik}]$ for $j \neq k$?
- **Lemma:** Let X and Y be two **independent** random variables. Then $\mathbb{E}[X \cdot Y] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.
- Since X_{ij} and X_{ik} are independent for $j \neq k$, we have $\mathbb{E}[X_{ij} \cdot X_{ik}] = \mathbb{E}[X_{ij}] \cdot \mathbb{E}[X_{ik}] = \frac{1}{n^2}$ for $j \neq k$.

Details of the Proof

Proving the Claim

- Let $X_{ij} = I\{A[j] \text{ falls in bucket } i\}$. Then, we have $n_i = \sum_{j=1}^n X_{ij}$.
- So, $\mathbb{E}[n_i^2] = \mathbb{E}\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] = \mathbb{E}\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right]$.
- $= \mathbb{E}\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} X_{ij} X_{ik}\right] = \sum_{j=1}^n \mathbb{E}[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} \mathbb{E}[X_{ij} X_{ik}]$.
- What is $\mathbb{E}[X_{ij}^2]$? $\mathbb{E}[X_{ij}^2] = 1 \cdot \frac{1}{n} + 0 \cdot \left(1 - \frac{1}{n}\right) = \frac{1}{n} \forall i, j$.
- What is $\mathbb{E}[X_{ij} X_{ik}]$ for $j \neq k$?
- **Lemma:** Let X and Y be two **independent** random variables. Then $\mathbb{E}[X \cdot Y] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.
- Since X_{ij} and X_{ik} are independent for $j \neq k$, we have $\mathbb{E}[X_{ij} \cdot X_{ik}] = \mathbb{E}[X_{ij}] \cdot \mathbb{E}[X_{ik}] = \frac{1}{n^2}$ for $j \neq k$.
- Hence, $\mathbb{E}[n_i^2] = \sum_{j=1}^n \frac{1}{n} + \sum_{1 \leq j \leq n} \sum_{1 \leq k \leq n, k \neq j} \frac{1}{n^2} = n \cdot \frac{1}{n} + n \cdot (n-1) \cdot \frac{1}{n^2} = 2 - \frac{1}{n}$ as claimed. □

Definition and Problem Statement

Order Statistic

- **Definition:** The i th **order statistic** of a set of n elements is the i th smallest element.

Definition and Problem Statement

Order Statistic

- **Definition:** The i th **order statistic** of a set of n elements is the i th smallest element.
- **Minimum** is the *first order statistic* ($i = 1$), **maximum** is the *n th order statistic* ($i = n$).

Definition and Problem Statement

Order Statistic

- **Definition:** The i th **order statistic** of a set of n elements is the i th smallest element.
- **Minimum** is the *first order statistic* ($i = 1$), **maximum** is the *n th order statistic* ($i = n$).
- A **median** is the "halfway point" of the set.

Definition and Problem Statement

Order Statistic

- **Definition:** The i th **order statistic** of a set of n elements is the i th smallest element.
- **Minimum** is the *first order statistic* ($i = 1$), **maximum** is the *n th order statistic* ($i = n$).
- A **median** is the "halfway point" of the set.
- When n is odd, the median is unique, occurring at $i = \frac{(n+1)}{2}$.

Definition and Problem Statement

Order Statistic

- **Definition:** The i th **order statistic** of a set of n elements is the i th smallest element.
- **Minimum** is the *first order statistic* ($i = 1$), **maximum** is the *n th order statistic* ($i = n$).
- A **median** is the "halfway point" of the set.
- When n is odd, the median is unique, occurring at $i = \frac{(n+1)}{2}$.
- When n is even, there are two medians. The **lower median** occurs at $i = \frac{n}{2}$, and the **upper median** occurs at $i = \frac{n}{2} + 1$.

Definition and Problem Statement

Order Statistic

- **Definition:** The i th **order statistic** of a set of n elements is the i th smallest element.
- **Minimum** is the *first order statistic* ($i = 1$), **maximum** is the *n th order statistic* ($i = n$).
- A **median** is the "halfway point" of the set.
- When n is odd, the median is unique, occurring at $i = \frac{(n+1)}{2}$.
- When n is even, there are two medians. The **lower median** occurs at $i = \frac{n}{2}$, and the **upper median** occurs at $i = \frac{n}{2} + 1$.
- We consistently use the phrase "the median" for the lower median, which occurs at $\lfloor \frac{(n+1)}{2} \rfloor$.

Definition and Problem Statement

Order Statistic

- **Definition:** The i th **order statistic** of a set of n elements is the i th smallest element.
- **Minimum** is the *first order statistic* ($i = 1$), **maximum** is the *nth order statistic* ($i = n$).
- A **median** is the "halfway point" of the set.
- When n is odd, the median is unique, occurring at $i = \frac{(n+1)}{2}$.
- When n is even, there are two medians. The **lower median** occurs at $i = \frac{n}{2}$, and the **upper median** occurs at $i = \frac{n}{2} + 1$.
- We consistently use the phrase "the median" for the lower median, which occurs at $\lfloor \frac{(n+1)}{2} \rfloor$.

The Selection Problem

- **Input:** A set A of n (distinct) numbers and a number i , with $1 \leq i \leq n$.
- **Output:** The element $x \in A$ that is larger than exactly $i - 1$ other elements of A .

Finding First Order Statistic

Pseudocode

```
MINIMUM(A)
    min = A[1];
    for i = 2 to length[A]
        if min > A[i]
            min = A[i];
    return min;
```

Finding First Order Statistic

Pseudocode

```
MINIMUM (A)
    min = A[1];
    for i = 2 to length[A]
        if min > A[i]
            min = A[i];
    return min;
```

Analysis of the Algorithm

- Who can give an invariant for the above loop?

Finding First Order Statistic

Pseudocode

```
MINIMUM (A)
    min = A[1];
    for i = 2 to length[A]
        if min > A[i]
            min = A[i];
    return min;
```

Analysis of the Algorithm

- Who can give an invariant for the above loop?
- Invariant:** At the start of the j th iteration of the loop, *min* keeps the smallest of the first j elements of A .

Finding First Order Statistic

Pseudocode

```
MINIMUM (A)
    min = A[1];
    for i = 2 to length[A]
        if min > A[i]
            min = A[i];
    return min;
```

Analysis of the Algorithm

- Who can give an invariant for the above loop?
- **Invariant:** At the start of the j th iteration of the loop, *min* keeps the smallest of the first j elements of A .
- What is the number of comparisons made?

Finding First Order Statistic

Pseudocode

```
MINIMUM (A)
    min = A[1];
    for i = 2 to length[A]
        if min > A[i]
            min = A[i];
    return min;
```

Analysis of the Algorithm

- Who can give an invariant for the above loop?
- **Invariant:** At the start of the j th iteration of the loop, min keeps the smallest of the first j elements of A .
- What is the number of comparisons made? Answer: $n - 1$.

Finding First Order Statistic

Pseudocode

```
MINIMUM (A)  
     $min = A[1];$   
    for  $i = 2$  to  $length[A]$   
        if  $min > A[i]$   
             $min = A[i];$   
    return  $min;$ 
```

Analysis of the Algorithm

- Who can give an invariant for the above loop?
- **Invariant:** At the start of the j th iteration of the loop, min keeps the smallest of the first j elements of A .
- What is the number of comparisons made? Answer: $n - 1$.
- Is it possible to find minimum with fewer comparisons?

Finding First Order Statistic

Pseudocode

```
MINIMUM (A)
    min = A[1];
    for i = 2 to length[A]
        if min > A[i]
            min = A[i];
    return min;
```

Analysis of the Algorithm

- Who can give an invariant for the above loop?
- Invariant:** At the start of the j th iteration of the loop, min keeps the smallest of the first j elements of A .
- What is the number of comparisons made? Answer: $n - 1$.
- Is it possible to find minimum with fewer comparisons? No. Each element except the minimum should be declared as larger than some element by a comparison. Thus, there is no algorithm that finds the minimum with fewer comparisons.

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!



Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!

Finding Order Statistic of a Given Element

- **Question:** If I give you an element of the array and ask you to find the order statistic of this element, can you find it in $O(n)$ -time?

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!

Finding Order Statistic of a Given Element

- **Question:** If I give you an element of the array and ask you to find the order statistic of this element, can you find it in $O(n)$ -time?
- **Answer:** Yes. By using the **PARTITION** procedure of **QUICKSORT**. We need to partition the array around the given element.

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!

Finding Order Statistic of a Given Element

- **Question:** If I give you an element of the array and ask you to find the order statistic of this element, can you find it in $O(n)$ -time?
- **Answer:** Yes. By using the **PARTITION** procedure of **QUICKSORT**. We need to partition the array around the given element.

Idea for Solving Selection Problem

- First, partition the array by using the **PARTITION** procedure.

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!

Finding Order Statistic of a Given Element

- **Question:** If I give you an element of the array and ask you to find the order statistic of this element, can you find it in $O(n)$ -time?
- **Answer:** Yes. By using the **PARTITION** procedure of **QUICKSORT**. We need to partition the array around the given element.

Idea for Solving Selection Problem

- First, partition the array by using the **PARTITION** procedure. If $i = q - p + 1$, we are done.

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!

Finding Order Statistic of a Given Element

- **Question:** If I give you an element of the array and ask you to find the order statistic of this element, can you find it in $O(n)$ -time?
- **Answer:** Yes. By using the **PARTITION** procedure of **QUICKSORT**. We need to partition the array around the given element.

Idea for Solving Selection Problem

- First, partition the array by using the **PARTITION** procedure. If $i = q - p + 1$, we are done. If $i < q - p + 1$, recurse on the left subarray. Else, recurse on the right subarray.

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!

Finding Order Statistic of a Given Element

- **Question:** If I give you an element of the array and ask you to find the order statistic of this element, can you find it in $O(n)$ -time?
- **Answer:** Yes. By using the **PARTITION** procedure of **QUICKSORT**. We need to partition the array around the given element.

Idea for Solving Selection Problem

- First, partition the array by using the **PARTITION** procedure. If $i = q - p + 1$, we are done. If $i < q - p + 1$, recurse on the left subarray. Else, recurse on the right subarray.
- What is the best case performance of this algorithm?

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!

Finding Order Statistic of a Given Element

- **Question:** If I give you an element of the array and ask you to find the order statistic of this element, can you find it in $O(n)$ -time?
- **Answer:** Yes. By using the **PARTITION** procedure of **QUICKSORT**. We need to partition the array around the given element.

Idea for Solving Selection Problem

- First, partition the array by using the **PARTITION** procedure. If $i = q - p + 1$, we are done. If $i < q - p + 1$, recurse on the left subarray. Else, recurse on the right subarray.
- What is the best case performance of this algorithm? $\Theta(n)$ since it may be the case that $i = q - p + 1$!

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!

Finding Order Statistic of a Given Element

- **Question:** If I give you an element of the array and ask you to find the order statistic of this element, can you find it in $O(n)$ -time?
- **Answer:** Yes. By using the **PARTITION** procedure of **QUICKSORT**. We need to partition the array around the given element.

Idea for Solving Selection Problem

- First, partition the array by using the **PARTITION** procedure. If $i = q - p + 1$, we are done. If $i < q - p + 1$, recurse on the left subarray. Else, recurse on the right subarray.
- What is the best case performance of this algorithm? $\Theta(n)$ since it may be the case that $i = q - p + 1$!
- What is the worst case running-time of this algorithm?

Other Simple Instances of Selection Problem

Finding Min and Max Together

- How many comparisons are sufficient to find the minimum and maximum?
- The trivial answer is $2n - 2$. But can we do better?
- We did that in class!

Finding Order Statistic of a Given Element

- **Question:** If I give you an element of the array and ask you to find the order statistic of this element, can you find it in $O(n)$ -time?
- **Answer:** Yes. By using the **PARTITION** procedure of **QUICKSORT**. We need to partition the array around the given element.

Idea for Solving Selection Problem

- First, partition the array by using the **PARTITION** procedure. If $i = q - p + 1$, we are done. If $i < q - p + 1$, recurse on the left subarray. Else, recurse on the right subarray.
- What is the best case performance of this algorithm? $\Theta(n)$ since it may be the case that $i = q - p + 1$!
- What is the worst case running-time of this algorithm? $\Theta(n^2)$! If you are always partitioning around the large or small elements. Why don't we use randomization then?

Randomized-Select Algorithm

Pseudocode

```
RANDOMIZED-SELECT(A, p, r, i)
    if p = r
        return A[p];

    q = RANDOMIZED-PARTITION(A, p, r);
    k = q - p + 1;

    if i = k
        return A[q];
    elseif i < k
        return RANDOMIZED-SELECT(A, p, q - 1, i);
    else
        return RANDOMIZED-SELECT(A, q + 1, r, i - k);
```

Expected Case Performance of Randomized-Select

Informal Argument

- **A trivial upper bound:** The algorithm is similar to **RANDOMIZED-QUICKSORT**. In **RANDOMIZED-QUICKSORT**, we recursively work on both subarrays. In **RANDOMIZED-SELECT**, we recursively work on only one side of the partition. Thus, expected running-time cannot be more than that of the **RANDOMIZED-QUICKSORT**. This argument gives us a $O(n \cdot \lg n)$ bound.

Expected Case Performance of Randomized-Select

Informal Argument

- **A trivial upper bound:** The algorithm is similar to **RANDOMIZED-QUICKSORT**. In **RANDOMIZED-QUICKSORT**, we recursively work on both subarrays. In **RANDOMIZED-SELECT**, we recursively work on only one side of the partition. Thus, expected running-time cannot be more than that of the **RANDOMIZED-QUICKSORT**. This argument gives us a $O(n \cdot \lg n)$ bound.
- **More involved bound:** From the analysis of **RANDOMIZED-QUICKSORT**, we know that **RANDOMIZED-PARTITION** partitions the array "somewhat balanced".

Expected Case Performance of Randomized-Select

Informal Argument

- **A trivial upper bound:** The algorithm is similar to **RANDOMIZED-QUICKSORT**. In **RANDOMIZED-QUICKSORT**, we recursively work on both subarrays. In **RANDOMIZED-SELECT**, we recursively work on only one side of the partition. Thus, expected running-time cannot be more than that of the **RANDOMIZED-QUICKSORT**. This argument gives us a $O(n \cdot \lg n)$ bound.
- **More involved bound:** From the analysis of **RANDOMIZED-QUICKSORT**, we know that **RANDOMIZED-PARTITION** partitions the array "somewhat balanced". Thus, the expected running time of **RANDOMIZED-QUICKSORT** is asymptotically equal to the solution of a recurrence of the form $T(n) = T(\frac{1}{a} \cdot n) + T(\frac{(a-1)}{a} \cdot n) + \Theta(n)$.

Expected Case Performance of Randomized-Select

Informal Argument

- **A trivial upper bound:** The algorithm is similar to **RANDOMIZED-QUICKSORT**. In **RANDOMIZED-QUICKSORT**, we recursively work on both subarrays. In **RANDOMIZED-SELECT**, we recursively work on only one side of the partition. Thus, expected running-time cannot be more than that of the **RANDOMIZED-QUICKSORT**. This argument gives us a $O(n \cdot \lg n)$ bound.
- **More involved bound:** From the analysis of **RANDOMIZED-QUICKSORT**, we know that **RANDOMIZED-PARTITION** partitions the array "somewhat balanced". Thus, the expected running time of **RANDOMIZED-QUICKSORT** is asymptotically equal to the solution of a recurrence of the form $T(n) = T(\frac{1}{a} \cdot n) + T(\frac{(a-1)}{a} \cdot n) + \Theta(n)$. Since in **RANDOMIZED-SELECT**, we recursively work on only one side of the partition, the expected running time is asymptotically equal to the solution of a recurrence of the form $T(n) = T(\frac{1}{a} \cdot n) + \Theta(n)$, which is $\Theta(n)$.

Select Algorithm

Idea

Since balanced partitioning gives good results, why don't we explicitly force balanced partitioning?

Select Algorithm

Idea

Since balanced partitioning gives good results, why don't we explicitly force balanced partitioning?

Algorithm Description

- Divide the n elements of the input array into $\lfloor \frac{n}{5} \rfloor$ groups of 5 elements each and at most one group made up of the remaining elements.

Select Algorithm

Idea

Since balanced partitioning gives good results, why don't we explicitly force balanced partitioning?

Algorithm Description

- Divide the n elements of the input array into $\lfloor \frac{n}{5} \rfloor$ groups of 5 elements each and at most one group made up of the remaining elements.
- Find the median of each of the $\lceil \frac{n}{5} \rceil$ groups by first insertion sorting the elements of each group and then picking the median from the sorted list of groups elements.

Select Algorithm

Idea

Since balanced partitioning gives good results, why don't we explicitly force balanced partitioning?

Algorithm Description

- Divide the n elements of the input array into $\lfloor \frac{n}{5} \rfloor$ groups of 5 elements each and at most one group made up of the remaining elements.
- Find the median of each of the $\lceil \frac{n}{5} \rceil$ groups by first insertion sorting the elements of each group and then picking the median from the sorted list of groups elements.
- Use **SELECT** recursively to find the median x of the $\lceil \frac{n}{5} \rceil$ medians found in the previous step.

Select Algorithm

Idea

Since balanced partitioning gives good results, why don't we explicitly force balanced partitioning?

Algorithm Description

- Divide the n elements of the input array into $\lfloor \frac{n}{5} \rfloor$ groups of 5 elements each and at most one group made up of the remaining elements.
- Find the median of each of the $\lfloor \frac{n}{5} \rfloor$ groups by first insertion sorting the elements of each group and then picking the median from the sorted list of groups elements.
- Use **SELECT** recursively to find the median x of the $\lfloor \frac{n}{5} \rfloor$ medians found in the previous step.
- Partition the input array around the median of medians x using "modified" Partition, which takes the pivot as parameter.

Select Algorithm

Idea

Since balanced partitioning gives good results, why don't we explicitly force balanced partitioning?

Algorithm Description

- Divide the n elements of the input array into $\lfloor \frac{n}{5} \rfloor$ groups of 5 elements each and at most one group made up of the remaining elements.
- Find the median of each of the $\lfloor \frac{n}{5} \rfloor$ groups by first insertion sorting the elements of each group and then picking the median from the sorted list of groups elements.
- Use **SELECT** recursively to find the median x of the $\lfloor \frac{n}{5} \rfloor$ medians found in the previous step.
- Partition the input array around the median of medians x using "modified" Partition, which takes the pivot as parameter.
- If $i = q - p + 1$ return x . If $i < q - p + 1$ recurse on the left subarray. Otherwise, recurse on the right subarray.

Analysis of Select

Worst-Case Analysis

- How many of the medians found in step 2 of the algorithm are larger than the median of the medians?

Analysis of Select

Worst-Case Analysis

- How many of the medians found in step 2 of the algorithm are larger than the median of the medians? At least half!
- Thus, at least half of the $\lceil \frac{n}{5} \rceil$ groups, except the group with fewer than 5 elements and the group of x has 3 elements larger than x .

Analysis of Select

Worst-Case Analysis

- How many of the medians found in step 2 of the algorithm are larger than the median of the medians? At least half!
- Thus, at least half of the $\lceil \frac{n}{5} \rceil$ groups, except the group with fewer than 5 elements and the group of x has 3 elements larger than x . Thus there are at least $3 \left(\lceil \frac{1}{2} \lceil \frac{n}{5} \rceil \rceil - 2 \right) \geq \frac{3 \cdot n}{10} - 6$ elements are larger than x .

Analysis of Select

Worst-Case Analysis

- How many of the medians found in step 2 of the algorithm are larger than the median of the medians? At least half!
- Thus, at least half of the $\lceil \frac{n}{5} \rceil$ groups, except the group with fewer than 5 elements and the group of x has 3 elements larger than x . Thus there are at least $3 \left(\lceil \frac{1}{2} \lceil \frac{n}{5} \rceil \rceil - 2 \right) \geq \frac{3 \cdot n}{10} - 6$ elements are larger than x .
- Similarly, there are at least $\frac{3 \cdot n}{10} - 6$ elements smaller than x .

Analysis of Select

Worst-Case Analysis

- How many of the medians found in step 2 of the algorithm are larger than the median of the medians? At least half!
- Thus, at least half of the $\lceil \frac{n}{5} \rceil$ groups, except the group with fewer than 5 elements and the group of x has 3 elements larger than x . Thus there are at least $3 \left(\lceil \frac{1}{2} \lceil \frac{n}{5} \rceil \rceil - 2 \right) \geq \frac{3 \cdot n}{10} - 6$ elements are larger than x .
- Similarly, there are at least $\frac{3 \cdot n}{10} - 6$ elements smaller than x . Thus, the last step of algorithm takes time at most $T(\frac{7 \cdot n}{10} + 6)$.

Analysis of Select

Worst-Case Analysis

- How many of the medians found in step 2 of the algorithm are larger than the median of the medians? At least half!
- Thus, at least half of the $\lceil \frac{n}{5} \rceil$ groups, except the group with fewer than 5 elements and the group of x has 3 elements larger than x . Thus there are at least $3 \left(\lceil \frac{1}{2} \lceil \frac{n}{5} \rceil \rceil - 2 \right) \geq \frac{3 \cdot n}{10} - 6$ elements are larger than x .
- Similarly, there are at least $\frac{3 \cdot n}{10} - 6$ elements smaller than x . Thus, the last step of algorithm takes time at most $T(\frac{7 \cdot n}{10} + 6)$.
- Since steps 1, 2, 4 take a total time of $O(n)$, and step 3 takes time $T(\lceil \frac{n}{5} \rceil)$, the running time of Select algorithm is bounded above by the recurrence
$$T(n) = T(\lceil \frac{n}{5} \rceil) + T(\frac{7 \cdot n}{10} + 6) + O(n).$$

Analysis of Select

Worst-Case Analysis

- How many of the medians found in step 2 of the algorithm are larger than the median of the medians? At least half!
- Thus, at least half of the $\lceil \frac{n}{5} \rceil$ groups, except the group with fewer than 5 elements and the group of x has 3 elements larger than x . Thus there are at least $3 \left(\lceil \frac{1}{2} \lceil \frac{n}{5} \rceil \rceil - 2 \right) \geq \frac{3 \cdot n}{10} - 6$ elements are larger than x .
- Similarly, there are at least $\frac{3 \cdot n}{10} - 6$ elements smaller than x . Thus, the last step of algorithm takes time at most $T(\frac{7 \cdot n}{10} + 6)$.
- Since steps 1, 2, 4 take a total time of $O(n)$, and step 3 takes time $T(\lceil \frac{n}{5} \rceil)$, the running time of Select algorithm is bounded above by the recurrence $T(n) = T(\lceil \frac{n}{5} \rceil) + T(\frac{7 \cdot n}{10} + 6) + O(n)$.
- **Exercise:** Show that the solution of the above recurrence is $O(n)$ by using the substitution method.